

4.4.2	Deploy Backend	23
4.4.3	Cấu hình App Service	23
5	Kiểm thử	24
5.1	Mục tiêu kiểm thử	24
5.2	Kiểm thử thủ công	24
5.2.1	Test Cases	24
5.3	Kiểm thử API bằng Swagger/Postman	25
5.4	Kiểm thử Giao diện	26
5.4.1	Kiểm thử Login	26
5.4.2	Kiểm thử Reset Password	26
5.4.3	Kiểm thử Admin Users	26
5.5	Vấn đề phát hiện và xử lý	27
5.5.1	Vấn đề 1: SPA reload trang con bị 404	27
5.5.2	Vấn đề 2: Dữ liệu bị reset sau restart	27
5.5.3	Vấn đề 3: Admin add-user không rõ lỗi	28
5.5.4	Vấn đề 4: Quên mật khẩu không gửi Gmail	28
6	Kết luận	29
6.1	Tóm tắt kết quả	29
6.2	Kỹ năng và công nghệ học được	29
6.3	Hạn chế	30
6.4	Hướng phát triển tương lai	30
6.5	Lời kết	30
A	Phụ lục A: Danh sách ảnh cần chèn	32
B	Phụ lục B: Code Snippets Chiều dài	33
B.1	Frontend - API Client Setup	33
B.2	Backend - Authentication Middleware	34
C	Phụ lục C: Các lệnh thường dùng	36

C.1	Frontend	36
C.2	Backend	36
D	Phụ lục D: Troubleshooting	38
D.1	Frontend	38
D.2	Backend	38

Danh sách hình vẽ

Danh sách bảng

Chương 1

Giới thiệu

1.1 Bối cảnh và lý do chọn đề tài

Trong bối cảnh chuyển đổi số tại các địa phương, nhu cầu cung cấp thông tin công khai và quảng bá du lịch ngày càng tăng. Thành phố Sa Đéc, một thành phố truyền thống giàu tiềm năng về du lịch và nền văn hóa, cần có một cổng thông tin định tập trung để quản lý các thông tin: tin tức địa phương, điểm đến du lịch, ẩm thực đặc sắc, và các sự kiện quan trọng.

Một hệ thống web hiện đại không chỉ giúp cung cấp thông tin kịp thời mà còn tạo cơ hội thu hút du khách, phát triển du lịch và nâng cao hình ảnh của địa phương trên các nền tảng kỹ thuật số.

1.2 Mục tiêu đề tài

Đề tài này hướng tới các mục tiêu sau:

1. Xây dựng website công khai (public) với giao diện thân thiện người dùng, hiển thị:
 - Tin tức địa phương (news).
 - Điểm đến du lịch (destinations).
 - Ẩm thực đặc sắc (cuisines).
 - Bản đồ hành chính và thông tin liên hệ.
 - Feedback từ khách hàng.
2. Xây dựng hệ thống quản trị (admin backend) cho phép:
 - Biên tập viên tạo, cập nhật, xoá nội dung.

- Quản lý trạng thái nội dung (draft, published, archived).
- Upload và quản lý hình ảnh/media.
- Duyệt bình luận và phản hồi.

3. Đảm bảo bảo mật:

- Xác thực người dùng bằng JWT (JSON Web Token).
- Phân quyền theo vai trò (Admin, Editor, Viewer).
- Bảo vệ API bằng CORS và authorization.

4. Triển khai trên nền tảng cloud (Azure):

- Frontend trên Azure Static Web Apps.
- Backend trên Azure App Service.
- Dữ liệu lưu trên Azure SQL Database.

5. Đảm bảo tính ổn định và tính sẵn sàng:

- Dữ liệu bền vững (persistent) trên Azure SQL.
- Tự động migration schema.
- Recovery mechanism cho các lỗi tạm thời.

1.3 Phạm vi thực hiện

Toàn bộ dự án bao gồm:

1.3.1 Frontend

- Trang chủ public với tính năng search.
- Trang chi tiết nội dung (news, destinations, cuisines).
- Trang login/đăng xuất.
- Dashboard admin với các bảng quản lý CRUD.
- Trang cài đặt hệ thống (logo, cấu hình).
- Trang quản lý người dùng.
- Trang quên/đặt lại mật khẩu.

1.3.2 Backend

- API public: lấy danh sách, chi tiết nội dung, cài đặt.
- API admin: CRUD nội dung, quản lý media, quản lý người dùng.
- API xác thực: login, refresh token, forgot/reset password.
- API quản lý cài đặt và log hành động admin.

1.3.3 Cơ sở dữ liệu

- Các bảng entity chính: Users, News, Destinations, Cuisines, Comments, Settings, MediaFiles.
- Audit log cho các hành động quan trọng.

1.4 Đối tượng sử dụng

- **Người dùng công khai (Public User):** Truy cập website để xem thông tin, tìm kiếm địa điểm, xem tin tức.
- **Biên tập viên (Editor):** Tạo, chỉnh sửa nội dung (news, destinations); không được quản lý người dùng.
- **Quản trị viên (Admin):** Toàn quyền: CRUD nội dung, quản lý người dùng, cài đặt hệ thống, xem log.

1.5 Môi trường triển khai

Hệ thống được triển khai trên các dịch vụ Azure:

Thành phần	Dịch vụ	URL/Endpoint
Frontend	Azure Static Web Apps	https://ashy-field-09c183d00.2.azurestaticapps.net
Backend API	Azure App Service	https://sadece-backend.azurewebsites.net
Swagger UI	App Service	https://sadece-backend.azurewebsites.net/swagger
Cơ sở dữ liệu	Azure SQL Database	sqlserver.database.windows.net

[CHÈN ẢNH]

Giao diện trang chủ

Tên file: ui_homepage.png

Nội dung: Chụp trang chủ sau deploy: banner, tin tức nổi bật, địa điểm nổi bật, bản đồ

Chương 2

Cơ sở lý thuyết

2.1 Kiến trúc ứng dụng web hiện đại

2.1.1 Mô hình Single Page Application (SPA)

Ứng dụng SPA là kiểu ứng dụng web tải toàn bộ code và resource một lần, sau đó điều hướng bằng JavaScript mà không cần tải lại trang. Ưu điểm:

- Trải nghiệm người dùng mượt mà (smooth UX).
- Giảm tải cho server (chỉ phục vụ API).
- Cho phép offline hoạt động (với service worker).

Thách thức:

- Reload trực tiếp route (ví dụ: `/admin/users`) có thể gặp lỗi 404 nếu server không cấu hình rewrite.
- Cần cấu hình CORS giữa frontend và backend nếu khác domain.

2.1.2 API RESTful

REST (Representational State Transfer) sử dụng HTTP method để biểu diễn các thao tác:

Method	Ý nghĩa	Endpoint ví dụ	Trả về
GET	Lấy dữ liệu	<code>/api/news/1</code>	200 OK + data
POST	Tạo mới	<code>/api/admin/news</code>	201 Created
PUT	Cập nhật toàn bộ	<code>/api/admin/news/1</code>	200 OK
PATCH	Cập nhật một phần	<code>/api/admin/news/1</code>	200 OK
DELETE	Xoá	<code>/api/admin/news/1</code>	204 No Content

2.2 Xác thực và phân quyền (Authentication & Authorization)

2.2.1 JWT (JSON Web Token)

JWT là chuỗi mã hóa gồm 3 phần (header.payload.signature), cho phép server xác minh danh tính của client mà không cần lưu trạng thái session.

Cấu trúc JWT:

Listing 2.1: Ví dụ JWT Payload

```
{
  "sub": "user_id",
  "email": "editor@sadec.local",
  "roles": ["Editor"],
  "exp": 1712345600,
  "iat": 1712342000
}
```

Luồng xác thực:

1. Client gửi username + password tới `/api/auth/login`.
2. Server trả access token (JWT) + refresh token.
3. Client gửi kèm header: `Authorization: Bearer <access_token>`.
4. Khi access token hết hạn, client dùng refresh token để lấy access token mới.

2.2.2 Role-Based Access Control (RBAC)

Hệ thống sử dụng 3 vai trò chính:

Vai trò	Quyền	API được phép
Admin	Toàn quyền	Tất cả endpoint
Editor	CRUD nội dung	<code>/api/admin/news/*</code> , <code>/api/admin/destinations/*</code>
Viewer	Chỉ đọc	<code>/api/public/*</code>

2.3 Công nghệ Stack

2.3.1 Frontend Stack

- **React 18:** Thư viện UI component-based.

- **TypeScript:** Tăng type-safety và đơn giản hóa debugging.
- **Vite:** Build tool nhanh, HMR (Hot Module Replacement) tức thời.
- **React Router v6:** Định tuyến SPA.
- **Axios:** HTTP client với interceptor.
- **Tailwind CSS:** Styling framework utility-first.

2.3.2 Backend Stack

- **ASP.NET Core 8:** Framework web API .NET.
- **ASP.NET Identity:** Quản lý user, role, password.
- **Entity Framework Core:** ORM cho SQL.
- **SQL Server / Azure SQL:** Cơ sở dữ liệu quan hệ.
- **Swagger:** Tài liệu API interactiv.

2.3.3 DevOps & Cloud

- **Azure Static Web Apps:** Triển khai frontend với CI/CD tự động.
- **Azure App Service:** Chạy backend ASP.NET.
- **Azure SQL Database:** Cơ sở dữ liệu cloud.
- **GitHub Actions:** Automation build và deploy.

2.4 Entity-Relationship Model (ERD)

2.4.1 Các bảng chính

- **AspNetUsers** (từ ASP.NET Identity):
 - Lưu user: Id, UserName, Email, PasswordHash, CreatedAt.
- **AspNetRoles:** Admin, Editor, Viewer.
- **News** (Tin tức):
 - Title, Slug, Content, Excerpt, FeaturedImageUrl.
 - Status (Draft, Published, Archived).

- CreatedBy, CreatedAt, UpdatedAt.
- **Destinations** (Địa điểm):
 - Name, Slug, Description, ImageUrl.
 - Coordinates (latitude, longitude).
 - Status, CreatedAt.
- **Cuisines** (Ẩm thực):
 - Name, Slug, Description, ImageUrl.
 - Status, CreatedAt.
- **Comments** (Bình luận):
 - Content, Status (Pending, Approved, Rejected).
 - EntityType (News, Destination), EntityId.
- **Settings** (Cài đặt hệ thống):
 - Key (ví dụ: site.logo, site.title, site.administrativeMapImageUrl).
 - Value (JSON).
- **AuditLog** (Log hành động):
 - Action, UserId, EntityType, EntityId, Timestamp.

[CHÈN ẢNH]

ERD (Entity Relationship Diagram)

Tên file: erd_{diagram}.png

Nội dung: Vẽ sơ đồ mối quan hệ các bảng hoặc chụp từ SSMS.

Chương 3

Phân tích và thiết kế

3.1 Khảo sát yêu cầu chi tiết

3.1.1 Yêu cầu chức năng (Functional Requirements)

Mã	Mô tả	Phạm vi
FR-01	Người dùng public xem danh sách tin tức (phân trang, search)	Public
FR-02	Người dùng public xem chi tiết tin tức	Public
FR-03	Người dùng public xem danh sách/chi tiết điểm đến	Public
FR-04	Người dùng public xem danh sách/chi tiết ẩm thực	Public
FR-05	Người dùng public gửi feedback	Public
FR-06	Xem bản đồ hành chính trên trang chủ	Public
FR-07	Đăng nhập hệ thống admin	Admin/Editor
FR-08	Quên mật khẩu và đặt lại (self-service)	Admin/Editor
FR-09	Tạo, chỉnh sửa, xoá tin tức	Admin/Editor
FR-10	Tạo, chỉnh sửa, xoá điểm đến	Admin/Editor
FR-11	Tạo, chỉnh sửa, xoá ẩm thực	Admin/Editor
FR-12	Upload, xoá hình ảnh/media	Admin/Editor
FR-13	Duyệt và phê duyệt bình luận	Admin
FR-14	Quản lý cài đặt hệ thống (logo, slogan, map)	Admin
FR-15	Quản lý người dùng (tạo, xoá, đổi role)	Admin
FR-16	Admin reset mật khẩu trực tiếp cho user	Admin
FR-17	Xem log hành động admin	Admin

Mã	Mô tả	Phạm vi
----	-------	---------

3.1.2 Yêu cầu phi chức năng (Non-Functional Requirements)

- **Bảo mật:**

- Tất cả API admin bắt buộc xác thực JWT.
- Mật khẩu mã hóa bcrypt.
- CORS cấu hình chặt chẽ.

- **Hiệu năng:**

- Danh sách dùng phân trang (mặc định 10-20 item).
- Media upload tối đa 50MB (tùy cấu hình).
- Response time < 2 giây cho API public.

- **Tính ổn định:**

- Dữ liệu bền vững (persistent) trên Azure SQL.
- Tự động migration schema khi update.
- Retry mechanism cho các lỗi tạm thời.

- **Khả năng triển khai:**

- CI/CD tự động qua GitHub Actions.
- Deploy frontend chỉ cần push code.
- Deploy backend bằng zip deployment đơn giản.

3.2 Use-Case Diagram

[CHÈN ẢNH]

Use-Case Diagram tổng quan

Tên file: usecase_diagram.png

Nội dung: Vẽ biểu đồ use-case: Public xem nội dung, Editor CRUD, Admin users settings

3.3 Thiết kế kiến trúc hệ thống

3.3.1 Sơ đồ kiến trúc 3-tier

Hệ thống theo kiến trúc 3 tầng:

1. Presentation Layer (Frontend):

- React SPA chạy trong trình duyệt.
- Gọi API backend qua Axios + interceptor.
- Quản lý state toàn cục (nếu cần dùng Context API hoặc Redux).

2. API/Business Logic Layer (Backend):

- ASP.NET Core Controllers xử lý request.
- Services chứa business logic.
- Middleware xử lý authentication, error handling.

3. Data Access Layer:

- Entity Framework Core ORM.
- Repositories pattern (tùy chọn).
- Azure SQL Database backend.

[CHÈN ẢNH]

Sơ đồ kiến trúc 3-tier

Tên file: `architecture3tier.png`

Nội dung: Vẽ hoặc chụp sơ đồ: Frontend SWA API App Service DB
Azure SQL

3.3.2 Cấu hình Static Web Apps cho SPA

Azure Static Web Apps yêu cầu cấu hình rewrite để hỗ trợ SPA routing:

Listing 3.1: Cấu hình SPA rewrite trong `staticwebapp.config.json`

```
{
```

```

"navigationFallback": {
  "rewrite": "/index.html",
  "exclude": ["/assets/*", "/api/*"]
}
}

```

Khi người dùng reload page `/admin/users`, server trả về `index.html`, đặt để React Router xử lý routing.

3.4 Thiết kế API

3.4.1 Base URL và cấu trúc endpoint

Nhóm	Base	Ví dụ endpoint
Public	<code>/api/public/</code>	GET <code>/api/news</code>
Admin	<code>/api/admin/</code>	POST <code>/api/admin/news</code>
Auth	<code>/api/auth/</code>	POST <code>/api/auth/login</code>
Settings	<code>/api/settings/</code>	GET <code>/api/settings</code>

3.4.2 Endpoint công khai (Public API)

Listing 3.2: Danh sách endpoint public

GET <code>/api/news?page=1&pageSize=10&q=search</code>	# ấly tin ứtc
GET <code>/api/news/{id}</code> tin ứtc	# Chi ếtit
GET <code>/api/news/slug/{slug}</code> slug	# Tìm theo
GET <code>/api/destinations?page=1&pageSize=10</code> địa đểim	# Danh sách
GET <code>/api/destinations/{id}</code> địa đểim	# Chi ếtit
GET <code>/api/cuisines?page=1&pageSize=10</code> ẩm ựthc	# Danh sách
GET <code>/api/settings</code> ổthng	# Cài đặ ệh
POST <code>/api/feedback</code> feedback	# ửGi

3.4.3 Endpoint admin (yêu cầu Bearer token + role Admin/Editor)

Listing 3.3: Danh sách endpoint admin

POST	/api/admin/news ớtc	# ặTo tin
GET	/api/admin/news?page=1&pageSize=20&status=1 admin	# Danh sách
PUT	/api/admin/news/{id} tin ớtc	# ặCp ậnh
DELETE	/api/admin/news/{id} ớtc	# Xóá tin
POST	/api/admin/destinations ểim	# ặTo ấia
PUT	/api/admin/destinations/{id}	# ặCp ậnh
DELETE	/api/admin/destinations/{id}	# Xóá
POST	/api/admin/media media	# Upload
GET	/api/admin/media/{id} file	# Chi ểtit
DELETE	/api/admin/media/{id}	# Xóá file
GET	/api/admin/users user	# Danh sách
POST	/api/admin/users	# ặTo user
PATCH	/api/admin/users/{id}/role	# Ớối role
DELETE	/api/admin/users/{id}	# Xóá user
POST	/api/admin/users/{id}/reset-password ấkhu	# Reset ậmt
GET	/api/settings settings	# ấLy
PUT	/api/settings/{key} setting	# ặCp ậnh
GET	/api/admin/audit-logs hành ộng	# Xem log

3.4.4 Endpoint xác thực (Auth API)

Listing 3.4: Endpoint auth

POST /api/auth/login	# Đăng nhập
POST /api/auth/refresh-token	# Cập token
POST /api/auth/forgot-password	# Quên mật
POST /api/auth/reset-password	# Đặt lại

3.4.5 Response Format

Tất cả API trả về JSON với cấu trúc chuẩn:

Listing 3.5: Response thành công với danh sách

```
{
  "items": [
    {"id": 1, "title": "Tin tức 1", "content": "..."},
    {"id": 2, "title": "Tin tức 2", "content": "..."}
  ],
  "page": 1,
  "pageSize": 10,
  "total": 25,
  "totalPages": 3
}
```

Listing 3.6: Response lỗi

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "Bad Request",
  "status": 400,
  "detail": "Title is required",
  "traceId": "..."
}
```

[CHÈN ẢNH]

Swagger UI - Tài liệu API interactive

Tên file: swagger_{ui}.png

Nội dung: Chụp Swagger UI trên máy production, hiển thị 3-4 endpoint chính.

3.5 Thiết kế giao diện (UI/UX)

3.5.1 Public Website

Trang công khai bao gồm:

- Trang chủ: banner, các tin/điểm đến nổi bật, bản đồ hành chính.
- Trang tin tức: danh sách phân trang, search, filter theo ngày.
- Trang chi tiết tin: toàn bộ nội dung, comments, share.
- Trang điểm đến: map tương tác, chi tiết địa điểm.
- Trang liên hệ / feedback form.

3.5.2 Admin Portal

Bao gồm:

- Dashboard: thống kê số tin, số user, hoạt động gần đây.
- Quản lý tin tức: bảng CRUD với search, filter theo status.
- Quản lý địa điểm: CRUD, upload hình.
- Quản lý âm thực: tương tự.
- Quản lý media: danh sách file, xóa, preview.
- Quản lý người dùng: bảng user, tạo, đổi role, reset password.
- Cài đặt hệ thống: form nhập key-value.
- Log hành động: bảng audit log chỉ xem.

[CHÈN ẢNH]

Trang admin dashboard

Tên file: ui_admin_dashboard.png

Nội dung: Chụp trang admin hiển thị thống kê, menu left sidebar

[CHÈN ẢNH]

Trang quản lý tin tức Admin

Tên file: ui_admin_news.png

Nội dung: Chụp bảng danh sách tin, nút thêm sửa xóa, filter status

[CHÈN ẢNH]

Modal tạo sửa tin tức

Tên file: ui_modal_news_edit.png

Nội dung: Chụp form: title, slug, content editor WYSIWYG status featured image

Chương 4

Cài đặt và triển khai

4.1 Cấu trúc mã nguồn

4.1.1 Frontend folder structure

Listing 4.1: Cấu trúc src/ frontend

```
src/|—
  app/| |—
    layouts/          # Layout component|—
    pages/            # Trang public + admin| |—
      public/         # Home, News, Destinations, ...| |—
      admin/          # Dashboard, AdminNews, AdminUsers, ...| |—
      auth/           # Login, ResetPassword, ...|—
    components/       # Component tái ử dụng|—
    services/         # API wrapper (*.service.ts)|—
    routes.tsx        # React Router config|—
    App.tsx|—
    main.tsx|—
  lib/| |—
    api.ts            # Axios instance + interceptor|—
    utils.ts|—
  styles/            # Global CSS|—
  types/             # TypeScript interfaces
```

4.1.2 Backend folder structure

Listing 4.2: Cấu trúc Sadec.Api/ backend

```

Sadec.Api/|—
  Controllers/          # API endpoints| |—
    AdminNewsController.cs| |—
    AdminUsersController.cs| |—
    AuthController.cs| |—
    DestinationsController.cs| |—
    NewsController.cs| |—
    ...| |—
  Dtos/                  # Request/Response DTO| |—
    AuthDtos.cs| |—
    NewsDtos.cs| |—
    UserAdminDtos.cs| |—
    ...| |—
  Entities/              # EF Core entity models| |—
    ApplicationUser.cs| |—
    News.cs| |—
    Destination.cs| |—
    ...| |—
  Data/| |—
    ApplicationDbContext.cs| |—
    DataSeeder.cs| |—
    Migrations/| |—
  Services/              # Business logic| |—
    Email/| |—
      IEmailSender.cs| |—
      SmtplibEmailSender.cs| |—
    AuditService.cs| |—
    ...| |—
  Middlewares/           # Custom middleware| |—
    ErrorHandlingMiddleware.cs| |—
    ...| |—
  Program.cs             # Startup config| |—
  appsettings.json       # Configuration| |—
  Sadec.Api.csproj

```

4.2 Setup môi trường local

4.2.1 Backend setup

Listing 4.3: Chạy backend local

```
# Clone repo
cd Sadec.Api

# Restore NuGet packages
dotnet restore

# Apply migrations (nếu DB chưa tạo)
dotnet ef database update

# chạy server
dotnet run --project ./Sadec.Api.csproj

# Swagger: http://localhost:5090/swagger
```

4.2.2 Frontend setup

Listing 4.4: Chạy frontend local

```
# Cài dependencies
npm ci

# chạy dev server
npm run dev

# Browser: http://localhost:5173
```

4.3 Cài đặt reset mật khẩu

Hệ thống hỗ trợ 2 luồng reset mật khẩu:

4.3.1 Luồng 1: Người dùng tự đặt lại (Forgot/Reset Password)

Backend implementation

Endpoint ‘/api/auth/forgot-password’:

Listing 4.5: Forgot Password Endpoint

```
[HttpPost]
```

```
[AllowAnonymous]
public async Task<IActionResult> ForgotPassword(
    [FromBody] ForgotPasswordRequestDto request)
{
    var user = await _userManager.FindByNameAsync(request.
        UsernameOrEmail)
        ?? await _userManager.FindByEmailAsync(request.
            UsernameOrEmail);

    if (user == null)
    {
        // Safety: không ếtit ột ảrng user không ồtn ạti
        return Ok(new {
            message = "If account exists, reset link sent to email"
        });
    }

    // Generate reset token
    var resetToken = await _userManager.
        GeneratePasswordResetTokenAsync(user);

    // Build reset URL
    var resetUrl = $"{Request.Scheme}://{Request.Host}/reset-password
        ?email={user.Email}&token={HttpUtility.UrlEncode(resetToken)}"
        ;

    // ửGi email (ếnu SMTP ấcu hình)
    if (_emailSender != null)
    {
        await _emailSender.SendPasswordResetEmailAsync(user.Email,
            resetUrl);
    }

    return Ok(new {
        message = "Reset link sent. Check email."
    });
}
```

Endpoint ‘/api/auth/reset-password’:

Listing 4.6: Reset Password Endpoint

```
[HttpPost]
```

```
[AllowAnonymous]
public async Task<IActionResult> ResetPassword(
    [FromBody] ResetPasswordRequestDto request)
{
    var user = await _userManager.FindByEmailAsync(request.Email);
    if (user == null)
        return BadRequest(new { message = "User not found" });

    var result = await _userManager.ResetPasswordAsync(
        user,
        request.Token,
        request.NewPassword);

    if (result.Succeeded)
        return Ok(new { message = "Password reset successful" });

    return BadRequest(new { message = string.Join(", ", result.Errors)
    });
}
```

Frontend implementation

Trang ‘/reset-password’:

Listing 4.7: URL format

```
/reset-password?email=user@sadec.local&token=ABC123XYZ
```

Component xử lý:

Listing 4.8: Logic

1. Parse URL query params (email, token)
2. Và form nhập mật khẩu mới + xác nhận
3. Gọi POST /api/auth/reset-password { email, token, newPassword }
4. Nếu thành công, chuyển hướng tới /login

4.3.2 Luồng 2: Admin reset mật khẩu trực tiếp

Endpoint ‘/api/admin/users/id/reset-password’:

Listing 4.9: Admin Reset Password

```
[HttpPost("{id}/reset-password")]
[Authorize(Roles = "Admin")]
public async Task<IActionResult> ResetPassword(
    string id,
    [FromBody] UserResetPasswordDto request)
{
    var user = await _userManager.FindByIdAsync(id);
    if (user == null)
        return NotFound();

    // Generate token mới
    var resetToken = await _userManager.
        GeneratePasswordResetTokenAsync(user);

    // Reset password
    var result = await _userManager.ResetPasswordAsync(
        user,
        resetToken,
        request.NewPassword);

    if (result.Succeeded)
    {
        // Log hành động
        _auditService.LogAction("user.password_reset_by_admin", user.
            Id);
        return NoContent(); // 204
    }

    return BadRequest(new { message = string.Join(", ", result.Errors
        ) });
}
```

Frontend component (Modal trong Admin Users page):

Listing 4.10: UI Modal reset

1. Chọn user từ bảng
2. Nhấn "Reset mật khẩu"
3. Modal yêu cầu: mật khẩu mới + xác nhận
4. Validation: độ dài, complexity
5. Gọi API + hiển thị kết quả

4.4 Triển khai trên Azure

4.4.1 Deploy Frontend

1. Build Vite: `npm run build` → output `dist/`
2. Copy `staticwebapp.config.json` vào `dist/`
3. Push code tới GitHub branch main
4. GitHub Actions workflow tự động deploy SWA
5. Xác minh: <https://ashy-field-09c183d00.2.azurestaticapps.net>

4.4.2 Deploy Backend

1. Build Release: `dotnet publish -c Release -o ./publish_backend`
2. Zip folder: `publish_backend` → `publish.zip`
3. Deploy via Azure CLI:

Listing 4.11: Deploy backend

```
az webapp deploy --resource-group <group> \
  --name <app-name> \
  --src-path ./publish.zip \
  --type zip
```

4. Xác minh: <https://sadec-backend.azurewebsites.net/swagger>

4.4.3 Cấu hình App Service

Các biến môi trường quan trọng (lưu trong App Service Configuration):

Key	Value ví dụ
DB_CONNECTION_STRING	Server=sql.database.windows.net;...
JWT_SECRET	your-secret-key-min-32-chars
CORS_ALLOWED_ORIGINS	https://ashy-field-09c183d00.2.azurestaticapps.net
SMTP_HOST	smtp.gmail.com (optional)
SMTP_PORT	587 (optional)
SMTP_USER	your-email@gmail.com (optional)
FRONTEND_BASE_URL	https://ashy-field-09c183d00.2.azurestaticapps.net

Chương 5

Kiểm thử

5.1 Mục tiêu kiểm thử

- Xác nhận các luồng chính hoạt động đúng.
- Xác nhận bảo mật: API admin bắt buộc xác thực, CORS hoạt động.
- Xác nhận dữ liệu bền vững: không bị reset khi restart.
- Xác nhận SPA routing không lỗi 404.

5.2 Kiểm thử thủ công

5.2.1 Test Cases

TC	Bước kiểm thử	Kỳ vọng	Kết quả
TC01	Truy cập trang chủ public	200 OK, hiển thị nội dung	PASS
TC02	Search tin tức	Danh sách tin phù hợp	PASS
TC03	Đăng nhập admin với credential đúng	Chuyển hướng /admin, nhận token	PASS
TC04	Đăng nhập admin với sai password	401 Unauthorized	PASS
TC05	Gọi API admin mà không token	401 Unauthorized	PASS
TC06	Gọi API admin với token nhưng role không đủ	403 Forbidden	PASS

TC	Bước kiểm thử	Kỳ vọng	Kết quả
TC07	Refresh token khi hết hạn	Tự động lấy token mới, retry thành công	PASS
TC08	Reload trang /admin/users	Không lỗi 404 (SPA routing hoạt động)	PASS
TC09	Tạo tin tức mới (admin)	Tin được lưu, xuất hiện trong danh sách admin	PASS
TC10	Xoá tin tức (admin)	Tin bị xoá, không còn hiển thị	PASS
TC11	Upload media (admin)	File lưu, URL trả về, download OK	PASS
TC12	Admin tạo user mới	User được tạo, có email + sinh password tạm	PASS
TC13	Admin reset password user	User đăng nhập được với password mới	PASS
TC14	Quên mật khẩu (không SMTP)	Endpoint trả 200 (tuy không gửi email)	PASS
TC15	Reset password bằng token	Mật khẩu update, đăng nhập với PW mới OK	PASS
TC16	Restart app, dữ liệu đã tạo vẫn có	Dữ liệu bền vững trên Azure SQL	PASS

5.3 Kiểm thử API bằng Swagger/Postman

Truy cập <https://sadece-backend.azurewebsites.net/swagger> để test interactive:

- Login endpoint: POST /api/auth/login
- Copy token từ response
- Click "Authorize" trong Swagger, paste token
- Test các endpoint admin (News, Users, Settings)

[CHÈN ẢNH]

Swagger Test Login endpoint

Tên file: `test_swagger_login.png`

Nội dung: Chụp Swagger request body response với token

[CHÈN ẢNH]

Swagger Test Admin Reset Password

Tên file: `test_swagger_admin_reset.png`

Nội dung: Chụp Swagger POST api admin users id reset-password trả 204

5.4 Kiểm thử Giao diện

5.4.1 Kiểm thử Login

- Input credential admin@sadec.local / Admin@123
- Xác nhận redirect /admin
- Xác nhận token lưu localStorage

5.4.2 Kiểm thử Reset Password

- Trang /reset-password?email=user@sadec.localtoken=XYZ
- Nhập password mới 2 lần
- Submit → xác nhận redirect /login
- Đăng nhập lại với password mới

5.4.3 Kiểm thử Admin Users

- Xem danh sách user (bảng phân trang)

- Nhấn "Thêm người dùng": modal mở
- Nhập email + password, nhấn "Tạo"
- Xác nhận user mới xuất hiện trong danh sách
- Nhấn "Reset mật khẩu" trên user: modal mở
- Nhập password mới, nhấn "Xác nhận"
- Xác nhận thành công / lỗi hiển thị rõ

[CHÈN ẢNH]

UI form Thêm người dùng Admin

Tên file: ui_form_add_user.png

Nội dung: Chụp modal email field password field nút Tạo

[CHÈN ẢNH]

UI modal Reset mật khẩu Admin Users

Tên file: ui_modal_reset_password.png

Nội dung: Chụp modal hiển thị user ô password mới ô xác nhận nút Xác nhận

5.5 Vấn đề phát hiện và xử lý

5.5.1 Vấn đề 1: SPA reload trang con bị 404

Nguyên nhân: Server không cấu hình rewrite cho SPA.

Giải pháp: Thêm navigationFallback trong staticwebapp.config.json.

Xác minh: Reload trang /admin/users → không 404, hiển thị bình thường.

5.5.2 Vấn đề 2: Dữ liệu bị reset sau restart

Nguyên nhân: Dùng InMemory Database.

Giải pháp: Chuyển sang Azure SQL + EF Core migrations.

Xác minh: Restart app, dữ liệu vẫn có.

5.5.3 Vấn đề 3: Admin add-user không rõ lỗi

Nguyên nhân: Frontend không đọc backend error message.

Giải pháp: Đổi service dùng shared API client, hiển thị `response.data.message`.

Xác minh: Tạo user với email trùng → hiển thị "Email already exists".

5.5.4 Vấn đề 4: Quên mật khẩu không gửi Gmail

Nguyên nhân: SMTP chưa cấu hình.

Giải pháp: Bổ sung admin reset password; SMTP là tùy chọn.

Xác minh: Endpoint trả 200; nếu có SMTP sẽ gửi email.

Chương 6

Kết luận

6.1 Tóm tắt kết quả

Dự án Cổng thông tin quảng bá Sa Đéc đã hoàn thành thành công với các thành tựu chính:

- **Hệ thống hoàn chỉnh:** Website public + admin portal với đầy đủ CRUD nội dung.
- **Bảo mật:** Xác thực JWT, phân quyền RBAC, CORS cấu hình chặt.
- **Tính bền vững:** Dữ liệu lưu Azure SQL, migrations tự động, không bị reset.
- **Triển khai thực tế:** Frontend trên SWA, backend trên App Service, CI/CD qua GitHub Actions.
- **Tính năng nâng cao:** Forgot/reset password, admin reset password, audit logging, settings management.
- **Quy trình phát triển:** Cấu trúc code rõ ràng, dễ bảo trì và mở rộng.

6.2 Kỹ năng và công nghệ học được

- **Frontend:** React, TypeScript, Vite, React Router, Axios interceptor.
- **Backend:** ASP.NET Core, EF Core, ASP.NET Identity, JWT authentication.
- **Cloud:** Azure Static Web Apps, Azure App Service, Azure SQL, deployment strategies.
- **DevOps:** GitHub Actions CI/CD, zip deployment, environment configuration.

6.3 Hạn chế

- SMTP email delivery chưa cấu hình (optional feature).
- Unit tests/integration tests còn hạn chế (có thể bổ sung thêm).
- Monitoring/logging advanced (Application Insights) chưa áp dụng.
- Performance optimization (caching, CDN) là hướng phát triển tiếp theo.

6.4 Hướng phát triển tương lai

1. **Hoàn thiện email:** Cấu hình SMTP + email templates chuyên nghiệp.
2. **Monitoring:** Tích hợp Application Insights, alert khi error.
3. **Caching:** Redis cache cho những query thường xuyên.
4. **Performance:** CDN cho static assets, database indexing tuning.
5. **Tính năng:** Two-factor authentication, OAuth social login.
6. **Testing:** Unit tests, integration tests, E2E tests.
7. **SEO:** Metadata, sitemap, structured data for content.

6.5 Lời kết

Đề tài này là một ứng dụng web thực tế, đã chuẩn bị theo quy trình DevOps chuyên nghiệp. Quá trình phát triển từ thiết kế, cài đặt, kiểm thử đến triển khai cloud đã minh chứng rõ ứng dụng của các kiến thức về web development, cloud computing, và software engineering.

Hy vọng báo cáo này cung cấp nền tảng vững chắc cho các nhân viên, teacher và các bên liên quan để hiểu rõ kiến trúc, công nghệ, và quy trình phát triển của hệ thống.

Tài liệu tham khảo

- [1] Microsoft, *ASP.NET Core Documentation*. <https://learn.microsoft.com/aspnet/core/>
- [2] Microsoft, *ASP.NET Core Identity Documentation*. <https://learn.microsoft.com/aspnet/core/security/authentication/identity>
- [3] Microsoft, *Entity Framework Core*. <https://learn.microsoft.com/ef/core/>
- [4] Facebook Developers, *React Official Documentation*. <https://react.dev/>
- [5] Microsoft, *Azure Static Web Apps*. <https://learn.microsoft.com/azure/static-web-apps/>
- [6] Microsoft, *Azure App Service Documentation*. <https://learn.microsoft.com/azure/app-service/>
- [7] Microsoft, *Azure SQL Database Documentation*. <https://learn.microsoft.com/azure/azure-sql/>
- [8] Auth0, *JWT.io - Introduction to JSON Web Tokens*. <https://jwt.io/introduction>
- [9] Roy Fielding, *Representational State Transfer (REST)*. https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

Phụ lục A

Phụ lục A: Danh sách ảnh cần chèn

Tên ảnh (figures/)	Nội dung gợi ý
<i>ui_homepage.png</i>	Trang chủ public: banner, tin nổi bật, bản đồ hành chính.
<i>erd_diagram.png</i>	Entity Relationship Diagram các bảng (SSMS hoặc draw.io).
<i>usecase_diagram.png</i>	Use-case diagram: Public actor, Editor, Admin.
<i>architecture_3tier.png</i>	Sơ đồ 3-tier: Frontend (SWA) -> API (App Service) -> DB.
<i>swagger_ui.png</i>	Swagger UI trên production: endpoint chính.
<i>ui_admin_dashboard.png</i>	Trang admin dashboard: menu, thống kê.
<i>ui_admin_news.png</i>	Bảng danh sách tin tức admin: header, data rows, action buttons.
<i>ui_modal_news_edit.png</i>	Modal tạo/sửa tin: form fields, buttons.
<i>ui_form_add_user.png</i>	Form thêm người dùng: email, password fields.
<i>ui_modal_reset_password.png</i>	Modal reset mật khẩu: user info, password fields, button.
<i>test_swagger_login.png</i>	Swagger: POST /api/auth/login + response.
<i>test_swagger_admin_reset.png</i>	Swagger: Endpoint reset password admin.

Phụ lục B

Phụ lục B: Code Snippets Chiều dài

B.1 Frontend - API Client Setup

Listing B.1: src/lib/api.ts - Axios Interceptor

```
import axios, { AxiosInstance } from "axios";

const API_BASE_URL =
  import.meta.env.VITE_API_URL ||
  (window.location.hostname.includes('azurestaticapps.net')
    ? 'https://sadec-backend.azurewebsites.net/api'
    : 'http://localhost:5000/api');

export const api = axios.create({
  baseURL: API_BASE_URL,
});

// Request interceptor: attach token
api.interceptors.request.use(config => {
  const token = localStorage.getItem("token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Response interceptor: refresh token on 401
api.interceptors.response.use(
  response => response,
  async error => {
```

```
const originalRequest = error.config;
if (error.response?.status === 401 && !originalRequest._retry) {
  originalRequest._retry = true;
  const refreshToken = localStorage.getItem("refreshToken");
  if (refreshToken) {
    const resp = await axios.post(`${API_BASE_URL}/auth/refresh-token`, {
      token: localStorage.getItem("token"),
      refreshToken
    });
    localStorage.setItem("token", resp.data.token);
    originalRequest.headers.Authorization = `Bearer ${resp.data.token}`;
    return api(originalRequest);
  }
}
return Promise.reject(error);
};
```

B.2 Backend - Authentication Middleware

Listing B.2: Program.cs - JWT Setup

```
var builder = WebApplicationBuilder.CreateBuilder(args);

// Add Authentication
builder.Services.AddAuthentication(JwtBearerDefaults.
  AuthenticationScheme)
  .AddJwtBearer(opts => {
    opts.TokenValidationParameters = new()
    {
      ValidateIssuer = false,
      ValidateAudience = false,
      ValidateLifetime = true,
      ValidateIssuerSigningKey = true,
      IssuerSigningKey = new SymmetricSecurityKey(
        Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Secret"] ?? "
      ))
    };
  });
```



```
// Add Authorization
builder.Services.AddAuthorization();

// Add DbContext
builder.Services.AddDbContext<ApplicationDbContext>(opts =>
    opts.UseSqlServer(builder.Configuration.GetConnectionString("
        DefaultConnection")));

var app = builder.Build();

app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();
app.Run();
```

Phụ lục C

Phụ lục C: Các lệnh thường dùng

C.1 Frontend

Listing C.1: Lệnh frontend

```
# Cài dependencies
npm ci

# ạy dev server
npm run dev

# Build production
npm run build

# Preview build
npm run preview

# Lint (nếu có ESLint)
npm run lint
```

C.2 Backend

Listing C.2: Lệnh backend

```
# Restore packages
dotnet restore

# Build
dotnet build
```

```
# ạyChy dev
dotnet run

# Publish Release
dotnet publish -c Release -o ./publish_backend

# EF Core migrations
dotnet ef migrations add <MigrationName>

# Apply migrations
dotnet ef database update

# View schema
dotnet ef dbcontext scaffold ...
```

Phụ lục D

Phụ lục D: Troubleshooting

D.1 Frontend

- **CORS error:** Backend cần cấu hình CORS cho origin của frontend.
- **Token không gửi:** Kiểm tra localStorage có token, axios interceptor chạy đúng.
- **404 khi reload route:** Kiểm tra staticwebapp.config.json có navigationFallback.

D.2 Backend

- **Connection string error:** Kiểm tra App Service configuration có DB connection string.
- **Migration fails:** Chắc chắn database đã tạo, firewall rule cho App Service.
- **401 Unauthorized:** Token hết hạn hoặc sai secret key.